Selenium Laboratory Experiments: Complete Guide

This document contains the setup instructions and Python code for all 10 Selenium laboratory experiments. Python is used as it is beginner-friendly and handles browser drivers automatically in its latest versions.

Part 1: Setup and Installation

Step 1: Download and Install Python

1. Go to python.org/downloads and download the latest version of Python.
2. Run the installer.
3. **CRITICAL:** Before clicking "Install Now", make sure you check the box that says **"Add Python to PATH"** at the bottom of the installation window.

Step 2: Install Selenium

1. Open your computer's Terminal (Mac) or Command Prompt (Windows).
2. Type the following command and press Enter:
   pip install selenium

Step 3: How to Run the Code

1. Create a new text file for each experiment.
2. Copy the code provided below for that specific experiment and paste it into the file.
3. Save the file with a .py extension (e.g., exp1.py).
4. **Run the file from your command prompt by typing: python exp1.py**
   (or simply double-click the file if your system is configured to run Python scripts).

Exp 1: Design and implement Selenium environment setup to configure WebDriver and verify browser.

```
from selenium import webdriver

# This automatically configures the WebDriver and opens Chrome
driver = webdriver.Chrome()

# Open a website
driver.get("[https://www.google.com](https://www.google.com)")

# Verify by printing the page title
print("Page Title is:", driver.title)

# Close the browser
driver.quit()
```

Exp 2: Design and implement basic browser navigation to automate opening, refreshing, and navigating pages.

```
from selenium import webdriver
import time

driver = webdriver.Chrome()

driver.get("[https://www.google.com](https://www.google.com)")
time.sleep(1) # Pausing so you can see it happen

driver.get("[https://www.wikipedia.org](https://www.wikipedia.org)") # Navigate to new page
time.sleep(1)

driver.back() # Go back to Google
time.sleep(1)

driver.forward() # Go forward to Wikipedia
time.sleep(1)

driver.refresh() # Refresh the page
print("Navigation successful!")

driver.quit()
```

Exp 3: Design and implement locators to find web elements using ID, XPath, or CSS selectors.

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("[https://the-internet.herokuapp.com/login](https://the-internet.herokuapp.com/login
)")

# 1. Find by ID
driver.find_element(By.ID, "username").send_keys("tomsmith")

# 2. Find by XPath
driver.find_element(By.XPATH, "//*[@id='password']").send_keys("SuperSecretPassword!")

# 3. Find by CSS Selector
driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()

driver.quit()
```

Exp 4: Design and implement form automation to fill and submit forms with test data.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()
driver.get("[https://the-internet.herokuapp.com/login](https://the-internet.herokuapp.com/login
)")

# Locate form elements
username_field = driver.find_element(By.ID, "username")
password_field = driver.find_element(By.ID, "password")
submit_btn = driver.find_element(By.CSS_SELECTOR, "button.radius")

# Fill data and submit
username_field.send_keys("tomsmith")
password_field.send_keys("SuperSecretPassword!")
```

```
submit_btn.submit() # Using the submit method directly

print("Form submitted!")
time.sleep(2)
driver.quit()
```

Exp 5: Design and implement dropdown and checkbox handling using Select and click methods.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select
import time

driver = webdriver.Chrome()

# Handle Dropdown using Select class
driver.get("[https://the-internet.herokuapp.com/dropdown](https://the-internet.herokuapp.com
/dropdown)")
dropdown = Select(driver.find_element(By.ID, "dropdown"))
dropdown.select_by_visible_text("Option 1")
time.sleep(1)

# Handle Checkbox using click method
driver.get("[https://the-internet.herokuapp.com/checkboxes](https://the-internet.herokuapp.co
m/checkboxes)")
checkbox = driver.find_element(By.XPATH, "//form[@id='checkboxes']/input[1]")
if not checkbox.is_selected():
    checkbox.click() # Click to check it

driver.quit()
```

Exp 6: Design and implement alert and pop-up handling to interact with JavaScript alerts.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()
driver.get("[https://the-internet.herokuapp.com/javascript_alerts](https://the-internet.herokuap
```

p.com/javascript_alerts)")

```
# Trigger the alert
driver.find_element(By.XPATH, "//button[text()='Click for JS Alert']").click()
time.sleep(1)

# Switch to the alert and accept it
alert = driver.switch_to.alert
print("Alert text says:", alert.text)
alert.accept() # Clicks "OK" on the alert

driver.quit()
```

Exp 7: Design and implement screenshot capturing to save images of web pages during testing.

```
from selenium import webdriver

driver = webdriver.Chrome()
driver.get("[https://www.google.com](https://www.google.com)")

# Capture and save screenshot to the same folder as this script
driver.save_screenshot("webpage_screenshot.png")
print("Screenshot saved successfully as 'webpage_screenshot.png'")

driver.quit()
```

Exp 8: Design and implement mouse and keyboard actions using the Actions class for advanced interactions.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains
from selenium.webdriver.common.keys import Keys
import time

driver = webdriver.Chrome()
actions = ActionChains(driver)

# Mouse Action (Hover)
driver.get("[https://the-internet.herokuapp.com/hovers](https://the-internet.herokuapp.com/ho
```

```
vers)")
avatar = driver.find_element(By.CLASS_NAME, "figure")
actions.move_to_element(avatar).perform() # Hovers mouse over image
time.sleep(1)

# Keyboard Action (Pressing Spacebar)
driver.get("[https://the-internet.herokuapp.com/key_presses](https://the-internet.herokuapp.com/key_presses)")
body = driver.find_element(By.TAG_NAME, "body")
actions.send_keys_to_element(body, Keys.SPACE).perform()

driver.quit()
```

Exp 9: Design and implement web table interaction to extract and process data from table elements.

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("[https://the-internet.herokuapp.com/tables](https://the-internet.herokuapp.com/tables)")

# Find the table rows
rows = driver.find_elements(By.XPATH, "//table[@id='table1']/tbody/tr")

print("Extracting Table Data:")
for row in rows:
    # Find all columns (td) in the current row
    cells = row.find_elements(By.TAG_NAME, "td")
    # Extract text from each cell
    row_data = [cell.text for cell in cells]
    print(row_data)

driver.quit()
```

Exp 10: Design and implement wait mechanisms to handle dynamic web elements using implicit or explicit waits.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
```

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()

# IMPLICIT WAIT: Tells driver to wait up to 10 seconds for ANY element to appear globally
driver.implicitly_wait(10)

driver.get("[https://the-internet.herokuapp.com/dynamic_loading/2](https://the-internet.herokuapp.com/dynamic_loading/2)")
driver.find_element(By.XPATH, "//div[@id='start']/button").click()

# EXPLICIT WAIT: Tells driver to halt and wait for a SPECIFIC condition to occur
wait = WebDriverWait(driver, 10) # Max 10 seconds wait
hidden_text = wait.until(EC.visibility_of_element_located((By.XPATH, "//div[@id='finish']/h4")))

print("Text loaded:", hidden_text.text)

driver.quit()
```